Guía de estudio — TP3, Ejercicio 4

Planificación Round Robin. Sistemas de Tiempo Real

1. Los números de memoria

Tarea	C (unid.)	T (unid.)	C (ms)	T (ms)
A	2	12	128	768
B	2	24	128	1536
C	2	6	128	384
D	1	3	64	192

Escala: 1 unidad = 4 ticks = **64 ms**. Hiperperíodo $\text{lcm}(12, 24, 6, 3) = 24$ unidades = **1536 ms**, con 22 unidades ocupadas y 2 ociosas.

Utilización: $U = 2/12 + 2/24 + 2/6 + 1/3 = \mathbf{0{,}9167}$.

Cuanto: 16 ms nominal, **16,05 ms** medido. Vuelta completa de las cuatro: **63,99 ms** $= 4 \times 16$.

Tick real: 94 ticks cada 3072 ms, o sea **32,7 ms** por tick contra los 16 que declara `portTICK_PERIOD_MS`.

Resultados: activaciones por reporte +4, +2, +8, +16.

Tarea	$n \cdot C$ teórico	`fin1`	$R_{\text{máx}}$ en 480 s	Jitter	% plazos perdidos
D	256	**257**	382	15	14,2 %
A	512	**507**	574	1	0
B	512	**523**	575	1	0
C	512	**529**	748	15	25,0 %

Ráfagas, sumadas tramo por tramo: A, B y C dan **128 ms** cada una repartidas en diez tramos; D da **64 ms** en cinco. Es la verificación del inciso (c) del Ejercicio 3.

2. La historia, en siete pasos

1. En el Ejercicio 3 se midió que durante una ráfaga de espera activa de 64 ms **no se registra ningún tick** (columna `dtT` = 0), cuando deberían registrarse cuatro. Con una sola tarea el desbalance se recuperaba durante el ocio y el período salía exacto.
2. Con las cuatro tareas el ocio ya no alcanza y el atraso se vuelve medible: el reporte imprime `xTaskGetTickCount()` al lado del reloj propio y da **94 ticks cada 3072 ms**, o sea 32,7 ms por tick. El tick corre a menos de la mitad de lo que supone la biblioteca.
3. El error está del lado del tick y no del reloj propio. El analizador mide 16,05 ms sobre un tramo que el reloj propio cuenta como 16: si el reloj propio corriera al doble, el analizador habría marcado 8.
4. Se descartó modificar el port. El banco se rediseñó para no depender del tick.
5. **Reloj propio:** un contador avanza 1 por cada milisegundo de CPU consumido, dentro del lazo calibrado. La tarea Idle consume el ocio de la misma forma, así que la suma mide tiempo real.
6. **Cuanto:** lo fuerza la tarea con `taskYIELD()`, que termina en `vTaskSwitchContext()` y rota la lista de listos igual que lo haría el tick.
7. **Activaciones:** las libera un despachador con notificaciones de tarea, llamado al cerrar cada cuanto, con el planificador suspendido para que las cuatro arranquen en el mismo instante.

3. La prueba de control

Es el argumento más fuerte que tiene el trabajo y conviene tenerlo listo, porque contesta de una sola vez la pregunta de si esto es Round Robin de verdad.

Se compiló el mismo firmware, con `configUSE_TIME_SLICING` en 1, quitando únicamente la llamada a `taskYIELD()`:

`fin1`	Con `taskYIELD()`	Sin `taskYIELD()`	Ejercicio 5 (`SLICING`=0)
B	523 ms	**143 ms**	143 ms
C	529 ms	**271 ms**	271 ms
D	257 ms	**335 ms**	335 ms
A	507 ms	**705 ms**	705 ms

Sin `taskYIELD()` el resultado es **idéntico al de FIFO**: ráfagas atómicas, mismos instantes, mismo orden, C con cero plazos perdidos y D con tres por hiperperíodo. O sea que en este simulador `configUSE_TIME_SLICING` no

tiene ningún efecto observable, porque la interrupción del tick no llega durante el cómputo y el núcleo nunca rota entre tareas de igual prioridad.

Dicho corto: *se cambió una sola variable y Round Robin se convirtió en FIFO*. Eso prueba que el `taskYIELD()` no es una comodidad sino la única razón por la que hay un Round Robin que medir.

4. Las dos cuentas que hay que saber hacer en el pizarrón

Tiempo de respuesta bajo reparto equitativo. Con n tareas listas, cada una avanza a $1/n$ de velocidad, así que tarda $R \approx n \cdot C$. Con $n = 4$: D tarda $4 \times 64 = 256$ y las otras $4 \times 128 = 512$.

Condición de cumplimiento.

$$n \cdot C_i \leq T_i \qquad \Longleftrightarrow \qquad \frac{C_i}{T_i} \leq \frac{1}{n}$$

La porción equitativa es 25 %. A pide 16,7 % y B pide 8,3 %: cumplen. C y D piden 33,3 %: pierden. Las cuatro predicciones se cumplen en la medición.

5. Preguntas probables y respuesta corta

- **¿Tocaste la biblioteca?** Solo `configUSE_TIME_SLICING` en `FreeRTOSConfig.h`, que es lo que el enunciado pide configurar, y queda en 1 para este ejercicio. Ningún otro archivo se modifica: no hay rastro de Timer1 ni CTC y `portUSE_WDTO` sigue en `WDTO_15MS`. El resto está en `main.cpp`, en un header del proyecto y en `platformio.ini`.
- **¿Cómo escribís los pines sin tocar la biblioteca?** FreeRTOS deja el gancho `traceTASK_SWITCHED_IN()` vacío detrás de un `#ifndef`. Si el proyecto lo define primero, la biblioteca se queda con esa definición y su archivo no cambia. El núcleo lo llama dentro de `vTaskSwitchContext()` y ahí se escribe el nibble bajo de PORTB entero.
- **¿Por qué no lo maneja la tarea?** Porque una tarea sabe cuándo empieza a usar el procesador pero no cuándo lo pierde. Si la expropian, se queda con el pin alto mientras otra ejecuta y el analizador muestra dos canales altos a la vez. Escribiéndolo desde el núcleo hay exactamente uno en alto, o ninguno si corre Idle.
- **¿Entonces esto es Round Robin de verdad?** La rotación sí: `taskYIELD()` llama a `vTaskSwitchContext()`, que toma el siguiente de la lista de listos, exactamente lo mismo que hace el tick con `configUSE_TIME_SLICING`. Lo que cambia es quién dispara el cambio de contexto: la tarea en vez de la interrupción. Y por qué hizo falta, está la prueba de control del punto 3.
- **¿Sigue siendo expropiativo?** Entre prioridades distintas sí: la tarea Idle es expropiada en cuanto se libera una periódica. Entre tareas de igual prioridad el reparto lo fuerza la tarea.
- **¿Por qué el cuanto vale 16 ms?** Para que sea el mismo valor que `portTICK_PERIOD_MS` y el cronograma sea comparable con el que daría un reparto por expropiación del tick.
- **¿Por qué hay tramos que no miden 16 ms?** Son los que el núcleo corta en los bordes donde el despachador reanuda el planificador. Ocurren cada 63 ms aproximadamente y siempre en parejas que suman un cuanto: A hace 14 y después 2, B hace 13 y después 3. El cuanto sigue siendo 16; a veces se entrega partido en dos.
- **¿Por qué se pierden plazos si $U < 1$?** Porque $U < 1$ garantiza que el trabajo entra, no cuándo entra. Round Robin reparte en partes iguales e ignora los plazos: C y D piden un tercio y reciben un cuarto.
- **¿Cuál es la más perjudicada?** C. Pierde el **25,0 %** de sus activaciones (313 de 1252, exactamente un plazo por hiperperíodo), contra el **14,2 %** de D (355 de 2504). Las dos piden lo mismo, pero D se activa el doble de veces y reparte una pérdida absoluta parecida entre el doble de trabajos.
- **¿Por qué $R_{\text{máx}}$ coincide con `fin1`?** En los primeros hiperperíodos sí, porque el peor caso ocurre en la primera ráfaga, con las cuatro tareas liberadas juntas: es el *instante crítico* del análisis de planificabilidad. Pero marca el peor caso *del arranque*, no el máximo absoluto: en 480 segundos los valores suben a 574, 575, 748 y 382 ms.
- **¿Por qué el jitter es 15 ms?** El despacho ocurre al cerrar cada cuanto, así que una activación no puede atrasarse más de un cuanto. 15 contra un techo de 16: la medición queda justo por debajo de lo que predice el modelo. A y B quedan en 1 porque sus activaciones son múltiplos de las de D y caen siempre en el mismo despacho que ella.
- **¿Por qué las cuatro compiten todo el tiempo?** D se reactiva a los 192 ms y recién termina su primera ráfaga a los 257: nunca sale de la lista de listos. Por eso A, B y C tardan 4×128 y no 3×128.
- **¿Cómo sabés que el reloj propio mide bien?** Dos vías. Las activaciones crecen +4, +2, +8, +16 entre reportes separados por dos hiperperíodos, y los totales de la corrida larga coinciden al entero con t/T_i. Y el cuanto que el reloj cuenta como 16 ms el analizador lo mide en 16,05.

- **¿Por qué el reloj propio va 8 % abajo del reloj de Proteus?** Porque cuenta milisegundos de CPU consumidos en el lazo calibrado, y el tiempo que la tarea Idle pasa bloqueada transmitiendo por el puerto serie no lo cuenta nadie: 156 bloques de reporte a 9600 baudios son unos 17 segundos. No afecta las medidas, porque los `fin1` ocurren en los primeros 530 ms, donde no se imprime nada.
- **¿Cómo detectás un plazo perdido?** Al liberar una activación, si la anterior sigue marcada como pendiente es que no terminó antes de la siguiente. Como $D = T$, eso es exactamente un plazo perdido.
- **¿Por qué el trabajo de fondo está en `loop()`?** Porque la biblioteca ejecuta `loop()` desde `vApplicationIdleHook()`, o sea desde la tarea Idle, que ya es la de menor prioridad y ya tiene su pila. Crear una sexta tarea agotaba el heap del ATmega328P.
- **¿Sirve la cota de Liu y Layland acá?** No. Esa cota es para prioridades fijas asignadas por período (Rate Monotonic). Round Robin no usa prioridades, así que la cota no dice nada sobre este caso. Lo que aplica es $C_i/T_i \leq 1/n$.

6. Las tres cosas que no hay que decir mal

- El tick **no** está detenido. Avanza, pero dura 32,7 ms en lugar de los 16 que declara la biblioteca, y no llega durante el cómputo. Son dos fallas distintas: como base de tiempo y como fuente de cuanto.
- El cuanto **no lo produce la interrupción del tick**. Coincide con el período nominal del tick porque se eligió así, no porque lo genere. Y que haga falta forzarlo está demostrado con la prueba de control.
- $U < 1$ **no garantiza** el cumplimiento de plazos bajo cualquier política. Solo dice que el trabajo total cabe en el procesador.

7. Cómo mostrarlo, en un minuto

1. Compilar y correr. La terminal arranca con `unidad=64ms cuanto=16ms politica=RR` y `PREEMPTION=1 TIME_SLICING=1`: ahí quedan fijadas la escala y la configuración que pide el enunciado, leída de la biblioteca.
2. Señalar el cronograma en texto y sumar en voz alta la ráfaga de D: $16 + 16 + 16 + 12 + 4 = 64$. Ese es el inciso (c) del Ejercicio 3 verificado.
3. Señalar la línea `tick=`: el contador por 16 da la mitad del reloj propio. Ahí está el problema del simulador en un solo número.
4. Esperar un bloque de reporte. Activaciones +4, +2, +8, +16; A y B con cero plazos perdidos; C y D perdiendo.
5. En el analizador, cursores sobre un tramo: **16,05 ms**. Sobre una vuelta completa: **63,99 ms**.
6. Cerrar con la columna `fin1`: 257 para D y 507, 523 y 529 para las otras tres, contra los 256 y 512 que predice $R = n \cdot C$.

8. Qué esperar en el Ejercicio 5 (FIFO)

Se cambia `configUSE_TIME_SLICING` a 0 y desaparecen los `taskYIELD()`: las ráfagas pasan a ser atómicas y `fin1` da 143, 271, 335 y 705 ms para B, C, D y A. La primera atendida mejora muchísimo y la última empeora.

Y ya lo sabemos de antemano, porque es exactamente lo que dio la prueba de control del punto 3.